

Week 40: From Logistic Regression to Our First Neural Network

Neurons, layers, activation functions – and backpropagation as the reverse sweep that does not stop

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 40

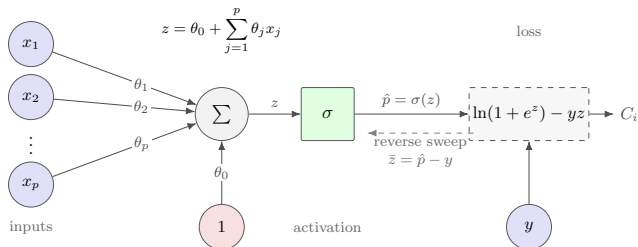
Monday lecture (2×45 min):

- ▶ **First 15 min, the reminder from last week:** logistic regression drawn as a single neuron and its learning rule (Section 5.8.1); the same picture read backwards – the computational graph, forward and reverse sweeps, and why every framework runs reverse mode (Section 5.8.2)
- ▶ **Neural networks and deep learning:** the artificial neuron, why one layer is not enough (XOR), multilayer perceptrons, the universal approximation theorem and what it does not say, activation functions, and the cost we have to minimise (Sections 8.1–8.7)
- ▶ **Backpropagation from the smallest networks:** one neuron with a scalar input, then one hidden neuron in front of it, by hand and with `jax.grad`; our first training program; then two inputs and two hidden nodes, derivative by derivative (Sections 8.8.1–8.8.2)
- ▶ **Next week:** the general case, the four equations of backpropagation and the algorithm, and how to write your own network code (Sections 8.8.3–8.8.5 and 8.13)

Tuesday (flipped classroom): your questions, then *only* Project 1 – two cases on the analysis and presentation of your results. **No weekly exercises this week:** the project is due **Monday October 5**.

Reading: Chapter 8, Sections 8.1–8.8; Section 5.8 once more. Goodfellow et al. Chapter 6; Raschka et al., *Machine Learning with PyTorch and Scikit-Learn*, Chapter 11.

Where we were: logistic regression as a single neuron (Section 5.8.1, Figure 5.4)



Eq. (5.35): $z = \theta_0 + \sum_j \theta_j x_j$, $\hat{p} = \sigma(z)$; the loss $\ln(1 + e^z) - yz$ of Eq. (5.14) is one more node.

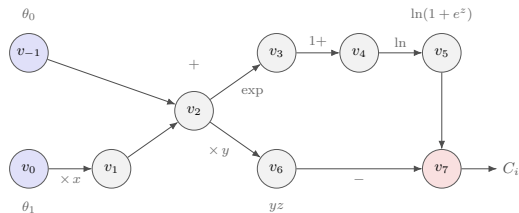
The vocabulary

x_j *inputs*; θ_j *weights*; θ_0 *bias* (the weight of an input fixed at one); z *pre-activation*; σ *activation function*. With the step function this is Rosenblatt's perceptron; with the identity it is linear regression.

The learning rule, Eqs. (5.36)–(5.37)

$\partial C_i / \partial \theta_j = (\hat{p}_i - y_i) x_{ij}$, so $\theta_j \leftarrow \theta_j - \gamma (\hat{p}_i - y_i) x_{ij}$: *error times the input on the edge*. The delta rule, SGD with $M = 1$ on the cross entropy; summed over i it is $X^T(p - y)$, Eq. (5.17).

The same picture, read backwards: the computational graph (Section 5.8.2)



Book Figure 5.5: the neuron redrawn as the evaluation trace of Section 4.14, with the parameters as *inputs*, the data x, y as constants on the edges, and the loss as output. Trace, Eq. (5.38): $v_{-1} = \theta_0$, $v_0 = \theta_1$, $v_1 = v_0 x$, $v_2 = v_1 + v_{-1} = z$, $v_3 = e^{v_2}$, $v_4 = 1 + v_3$, $v_5 = \ln v_4$, $v_6 = y v_2$, $v_7 = v_5 - v_6 = C_i$.

Two readings of one diagram

As a **model**: data flow left to right, parameters are fixed numbers on the edges, a probability comes out.
As a **computational graph**: same nodes, same edges, roles exchanged – we differentiate with respect to the parameters, the data are constants. The one structural feature: $z = v_2$ fans out into the softplus branch and the label branch.

Local derivatives, Eq. (5.39) – eight numbers

$\partial v_1 / \partial v_0 = x$, $\partial v_2 / \partial v_1 = \partial v_2 / \partial v_{-1} = 1$,
 $\partial v_3 / \partial v_2 = v_3$, $\partial v_4 / \partial v_3 = 1$, $\partial v_5 / \partial v_4 = 1/v_4$,
 $\partial v_6 / \partial v_2 = y$, $\partial v_7 / \partial v_5 = 1$, $\partial v_7 / \partial v_6 = -1$.
Everything that follows combines these.

Forward sweep, reverse sweep – with numbers (Eqs. 5.40–5.41)

Take $\theta = (0.5, 0.25)$, $x = 2$, $y = 1$: the tape holds $z = 1$, $v_3 = 2.718$, $v_4 = 3.718$, $v_5 = 1.313$, $C_i = 0.313$, $p = v_3/v_4 = 0.731$.

Forward mode: seed $(\dot{v}_{-1}, \dot{v}_0) = (1, 0)$,
Eq. (5.40)

$\dot{v}_1 = x\dot{v}_0 = 0$, $\dot{v}_2 = 1$, $\dot{v}_3 = v_3$, $\dot{v}_4 = v_3$,
 $\dot{v}_5 = v_3/v_4 = p$, $\dot{v}_6 = y$, $\dot{v}_7 = p - y = -0.269$.
That is $\partial C_i / \partial \theta_0$ – and only that. Seed $(0, 1)$ and sweep
again: every tangent is the one above times x , ending
in $x(p - y) = -0.538$. **Two parameters, two sweeps;**
 p parameters, p sweeps.

Reverse mode: $\bar{v}_7 = 1$, back along the
tape, Eq. (5.41)

$\bar{v}_5 = 1$, $\bar{v}_6 = -1$, $\bar{v}_4 = 1/v_4$, $\bar{v}_3 = 1/v_4$,
 $\bar{v}_2 = \bar{v}_3 v_3 + \bar{v}_6 y = v_3/v_4 - y = p - y$ (the fan-out
sums),
 $\bar{v}_1 = \bar{v}_2$, $\bar{v}_{-1} = \bar{v}_2 = p - y = -0.269$,
 $\bar{v}_0 = \bar{v}_1 x = x(p - y) = -0.538$.
One sweep, both derivatives, read off at the input
nodes.

What the reverse sweep has produced

At v_2 the two branches contribute p and $-y$: the *residual* $\hat{p} - y$, which Section 5.4 obtained by differentiating the log-odds by hand, appears as the sensitivity of the cost to the linear predictor, produced mechanically by the fan-out rule. From v_2 down to the parameters the sweep only multiplies by the input on each edge, 1 and x – the delta rule, done by the machine. Keep the name: **the adjoint of the pre-activation**, $\bar{z}$. Today it will be called δ .

The whole data set, and why every framework runs reverse mode (Eq. 5.42)

The cost is $\sum_i C_i$: n copies of the graph hanging from the *same* p parameter nodes, feeding one summation node.

- **Reverse:** the sum passes $\bar{C}_i = 1$ to every copy, each produces its residual $r_i = p_i - y_i$ at z_i , each parameter node accumulates $\sum_i X_{ij} r_i$:

$$\bar{\theta} = X^T(p - y)$$

in *one* pass, $\mathcal{O}(np)$ – one $X\theta$ forward, one $X^T r$ back. Tape: the n numbers z_i .

- **Forward:** seed e_j , get one component $\sum_i X_{ij} r_i$; p sweeps, $\mathcal{O}(np^2)$.

Measured (book Section 5.8.2, JAX, $n = 5000$)

p	value	reverse	forward
20	0.13 ms	0.19 ms	0.41 ms
200	0.32 ms	0.47 ms	4.53 ms
1000	1.59 ms	3.31 ms	85.9 ms

Reverse: 1.4–2 evaluations of C whatever p – the cheap gradient principle, Prop. 4.2. Forward: 3, 14, 54 evaluations, growing with p . Both agree with $X^T(p - y)$ to 10^{-16} .

The rule

Forward mode: one input direction to all outputs. Reverse mode: one output to all inputs. A cost has one output and p inputs – training asks the reverse-mode question. Price: the tape. A network with 10^6 weights asks it 10^6 times more loudly.

Code: the trace, both sweeps, and JAX (Section 5.8.2; week40.ipynb)

```
def cross_entropy_one(theta0, theta1, x, y): # the trace of Eq. (5.38), one node per line
    v1 = theta1 * x; v2 = v1 + theta0; v3 = jnp.exp(v2); v4 = 1.0 + v3
    v5 = jnp.log(v4); v6 = y * v2
    return v5 - v6

def reverse_sweep(theta0, theta1, x, y): # Eq. (5.41) by hand
    v1 = theta1 * x; v2 = v1 + theta0; v3 = np.exp(v2); v4 = 1.0 + v3 # forward, onto the tape
    b7 = 1.0; b5, b6 = b7, -b7; b4 = b5 / v4; b3 = b4
    b2 = b3 * v3 + b6 * y # fan-out: two successors, one sum
    return b2, b2 * x # dC/dtheta0, dC/dtheta1

cost_one = lambda th: cross_entropy_one(th[0], th[1], 2.0, 1.0)
theta = jnp.array([0.5, 0.25])
for k in range(2): # forward mode: one sweep per parameter
    _, tangent = jax.jvp(cost_one, (theta,), (jnp.eye(2)[k],))
print(jax.grad(cost_one)(theta)) # reverse mode: one sweep, both -> [-0.269 -0.538]

def cost(theta, X, y): # the whole data set, Eq. (5.43)
    z = X @ theta
    return jnp.mean(jnp.logaddexp(0.0, z) - y * z)
grad_ad = jax.grad(cost)(theta_p, X, y) # == X.T @ (sigmoid(X @ theta_p) - y) / n
```

`jax.grad(cost)` is the gradient of Eq. (5.17). Hold on to the pattern: we write the forward pass, the library writes the backward one. In fifty minutes we will have written the backward pass ourselves for a network – and checked it against this.

Pause and think 1: one layer more

Question

Suppose the inputs x_j of the neuron in Figure 5.4 were not the raw features but the outputs of another layer of such neurons, $x_j = \sigma(\mathbf{w}_j^T \mathbf{u} + b_j)$. What does the reverse sweep do when it reaches $\bar{z} = \hat{p} - y$ – and what is the name of the algorithm?

Pause and think 1: one layer more

Question

Suppose the inputs x_j of the neuron in Figure 5.4 were not the raw features but the outputs of another layer of such neurons, $x_j = \sigma(w_j^T u + b_j)$. What does the reverse sweep do when it reaches $\bar{z} = \hat{p} - y$ – and what is the name of the algorithm?

Answer

It does not stop. From $\bar{z}$ it walks down each edge to the parameter it carries – $\bar{\theta}_j = \bar{z} x_j$, the delta rule – but also to the *input* x_j of that edge: $\bar{x}_j = \bar{z} \theta_j$. Since x_j is itself the output of a neuron, the sweep continues through its activation, $\bar{z}_j = \bar{x}_j \sigma'(z_j)$, and then along its edges to w_j and to the layer before. Layer by layer, the adjoint of each layer's output is multiplied by the transposed weight matrix and by the derivative of the activation: that is **backpropagation**, and Eq. (5.41) is its one-layer special case. The $\sigma' \leq 1/4$ factor is multiplied in once per layer – the reason deep sigmoid networks stop learning, and the reason the activation function is chosen with care.

The artificial neuron (Section 8.1, book Figure 8.1)

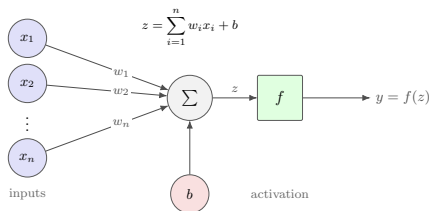
Eq. (8.1): n inputs, a weighted sum, a bias, a non-linearity,
 $y = f(\sum_{i=1}^n w_i x_i + b) = f(z)$.

Nothing new – three models under one name

f = identity: linear regression (Chapter 3). $f = \sigma$: logistic regression, exactly (Chapter 5). f = step: Rosenblatt's perceptron (1958), the first machine-learning model.

What changes now

In a *network* the inputs of one neuron are the *outputs* of the neurons in the layer before. **Fully connected**: every neuron sees every output of the previous layer. **Feed-forward**: information flows one way, no cycles. The features fed to the final sigmoid are then *learned*, not given – that is the whole point, and the whole price: no convexity, no closed form, only the chain rule.



Why one layer is not enough: XOR (Section 8.2, book Figure 8.2)

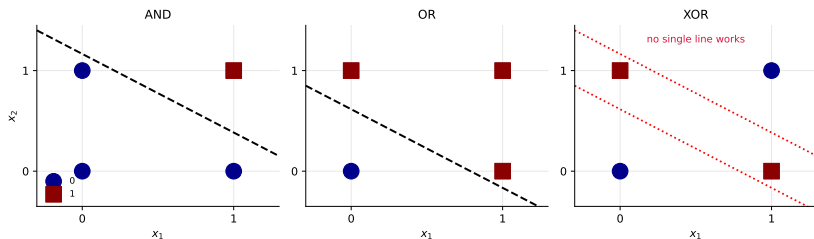
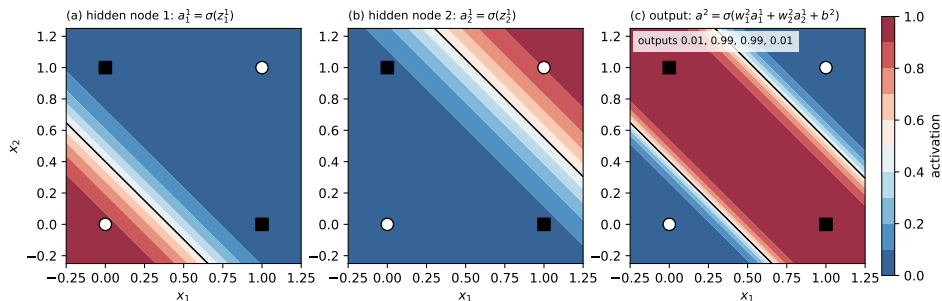


Table 8.1: AND, OR and XOR of two binary inputs. Least squares on XOR returns $\theta = (0.5, 0, 0)$ – the best *linear* function is the constant $1/2$; logistic regression scores 0.50, chance. Not a training failure: **the model class contains no solution**, because one neuron computes one straight line and no line puts both circles on one side. This stalled the field for two decades (Minsky and Papert, 1969).

The remedy

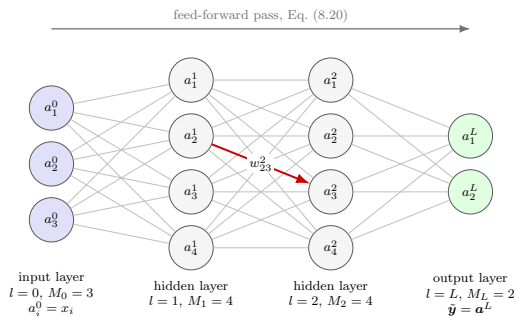
Two hidden neurons form two intermediate features – “ x_1 or x_2 ” and “not both” – and the output neuron combines *them* linearly: accuracy 1.00 on all three gates with the class of Section 8.13. Seen from Chapter 3: a basis expansion whose basis functions are *fitted*, not chosen.

What a hidden layer supplies: the features it learned (week40.ipynb)



The 2–2–1 network we derive in Part 3 (sigmoid hidden nodes, sigmoid output, squared error, plain gradient descent, $\gamma = 2$, 20 000 iterations, seed 2026), trained on the four XOR points. Each hidden node is one neuron and draws one line: node 1 fires only near (0,0) (“neither”), node 2 only near (1,1) (“both”). In the plane of (a_1^1, a_2^1) the four points are linearly separable, and the output neuron is the line “not neither and not both”: outputs 0.01, 0.99, 0.99, 0.01. A single neuron trained the same way returns 0.5 four times. **Non-linearity is essential:** with linear hidden units $W^2(W^1x + b^1) + b^2$ is again affine in x – no depth of linear layers escapes linearity.

Multilayer perceptrons and other architectures (Section 8.3, Figure 8.5)

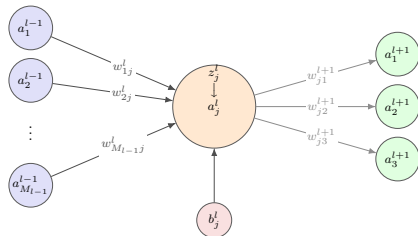


A fully connected feed-forward network with an input layer, one or more hidden layers and an output layer, non-linear activations throughout: the **multilayer perceptron** (MLP) – the architecture we analyse, and the one for which backpropagation is simplest. Here $L = 3$: $M_0 = 3$ inputs, two hidden layers of four, two outputs.

Other architectures (later chapters)

Convolutional: a small kernel slid across the input, weights shared across positions – translation invariance, far fewer parameters; images. **Recurrent**: the output of a layer fed back at the next time step – a memory; sequences. **Autoencoders**: a bottleneck between encoder and decoder, trained to reproduce the input – non-linear PCA.

Notation and the feed-forward pass (Section 8.5, book Figure 8.6)



$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \quad a_j^l = f(z_j^l)$$

Layers $l = 0, 1, \dots, L$ with M_l nodes; $a_i^0 = x_i$, $\tilde{y} = a^L$. Node j of layer l , Eqs. (8.18)–(8.19):

$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \quad a_j^l = f(z_j^l).$$

w_{ij}^l : superscript = the layer it *arrives* at, first index = the node it *leaves*, second = the node it arrives at; the forward sum runs over the first index, the backward sum (next week) over the second. Almost every difficulty with backpropagation is a difficulty with indices.

The feed-forward pass, Eqs. (8.20)–(8.21)

$z^l = (W^l)^T a^{l-1} + b^l$, $a^l = f(z^l)$ for $l = 1, \dots, L$; with n samples as rows, $Z^l = A^{l-1} W^l + b^l$ – the form our code uses ($X @ W1 + b1$), W^l untransposed; mixing the two is the most frequent shape error. Cost $\mathcal{O}(\sum_l M_{l-1} M_l)$ = the number of weights: level-3 BLAS interleaved with element-wise f .

Pause and think 2: count the parameters, and the functions

Question

A network for the 28×28 digit images has 784 inputs, two hidden layers of 100 nodes and 10 outputs. How many parameters does it have? If the hidden activations were *linear*, how many numbers would it take to write down every function it can compute – and what does that say about the 100 hidden nodes?

Pause and think 2: count the parameters, and the functions

Question

A network for the 28×28 digit images has 784 inputs, two hidden layers of 100 nodes and 10 outputs. How many parameters does it have? If the hidden activations were *linear*, how many numbers would it take to write down every function it can compute – and what does that say about the 100 hidden nodes?

Answer

Weights and biases: $784 \cdot 100 + 100 + 100 \cdot 100 + 100 + 100 \cdot 10 + 10 = 89\,610$. With linear hidden layers the whole network is one affine map $x \mapsto Ax + c$ with $A = W^1 W^2 W^3$ of size 784×10 : at most 7 850 numbers matter, and since $\text{rank } A \leq 100$ is no restriction for a 784×10 matrix, it is exactly the multinomial logistic regression of Section 5.6 with an 89 610-parameter way of writing 7 850 numbers. The hidden nodes buy nothing. Everything the extra 81 760 parameters can do, they do through the non-linearity of f – which is why the next frames are about which f , and why the theorem after them requires f not to be a polynomial.

The universal approximation theorem (Section 8.4)

Cybenko (1989), Eq. (8.3)

σ continuous and sigmoidal ($\rightarrow 1$ at $+\infty$, $\rightarrow 0$ at $-\infty$), F continuous on $[0, 1]^d$. For every $\epsilon > 0$ there is a *one-hidden-layer* network with

$$|F(x) - f(x; \Theta)| < \epsilon \quad \text{for all } x \in [0, 1]^d.$$

Same shape of argument as Stone–Weierstrass for polynomials and the convergence of Fourier series: a family of simple functions is dense. To be regarded with the same mixture of reassurance and scepticism.

Hornik (1991), Eq. (8.5)

Activation only non-constant and bounded, and the statement in mean square, $\mathbb{E}|F - f|^2 < \epsilon$ over the input density – the expected squared error of Section 2.10, not a worst case over a cube the data never visit. Universality is a property of the *architecture*, not of sigmoids.

Leshno, Lin, Pinkus, Schocken (1993): the sharp condition

One hidden layer is a universal approximator **if and only if the activation is not a polynomial** on any interval. So ReLU qualifies although unbounded; every function of Section 8.6 qualifies; the *one* that fails is the linear activation – Pause and think 2, recovered from the general theory.

A constructive version, Theorem 8.2, Eq. (8.7)

In one dimension, F L -Lipschitz on $[a, b]$: N rectified units give $\sup |F - g| \leq L(b - a)/2N$ – the network *is* the piecewise-linear interpolant at $N + 1$ knots, and only the $N + 1$ output weights depend on F . The classical proofs give no such relation between ϵ and the width.

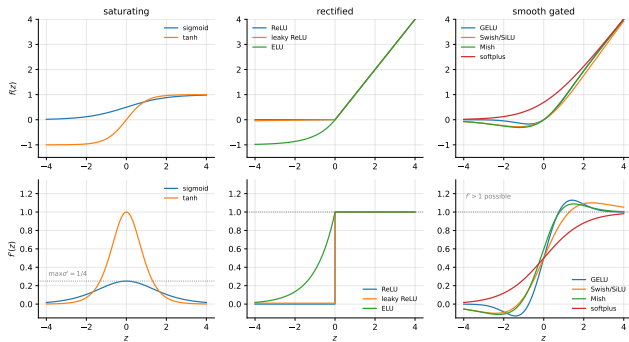
What the theorem does not say (Section 8.4)

1. **Hidden layer only.** The output nodes are linear; a sigmoid output could never represent the value 7. The output activation is dictated by the range of the target, the loss by its distribution (Section 3.6) – never chosen independently.
2. **Existence, not construction.** The classical proofs relate ϵ to nothing – not to the width (which may grow exponentially with d) and not to the size of the weights – and say nothing about whether gradient descent on a non-convex cost will *find* the weights. *Representability is not learnability*; the distance between the two is where the practical difficulty of deep learning lives (Section 8.4.4).
3. **Universality is not remarkable.** Polynomials, Fourier series, splines and radial basis functions are universal too. What distinguishes networks is how *economically* they approximate in high dimensions, and that *depth* beats width: Theorem 8.6 exhibits a function that a deep network represents with a number of units polynomial in the depth and a shallow one needs exponentially many for. The theorem that names the field is about one hidden layer – precisely the case it does not cover.

Hold it next to Chapter 3

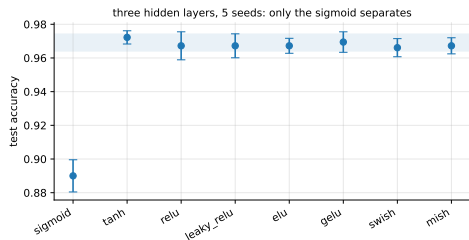
Polynomials are universal, and the degree-14 fit of Figure 3.7 failed catastrophically – not because f was unrepresentable but because the coefficients could not be *estimated* from a hundred noisy points. Universality is a statement about the model class; generalisation is about the class, the data and the estimator together, Eq. (2.52). A network large enough to approximate anything is large enough to overfit anything.

Activation functions: three families – only the derivative matters (Section 8.6)



Saturating, Eq. (8.22): $\sigma(z)$ with $\sigma' = \sigma(1 - \sigma) \leq 1/4$; $\tanh(z) = 2\sigma(2z) - 1$, centred, $\tanh' \leq 1$ – both die in the tails. **Rectified**, Eqs. (8.23)–(8.25): $\text{ReLU}(z) = \max(0, z)$, derivative exactly 1 or 0 – cheap, non-saturating, sparse; its failure is the *dying* unit (input negative for every sample $\Rightarrow$ zero gradient forever); leaky ReLU (αz for $z < 0$), ELU, SELU repair the negative branch. **Smooth gated**, Eqs. (8.26)–(8.28): $\text{GELU}(z) = z\Phi(z)$, Swish $z\sigma(z)$, Mish – non-monotone, derivative slightly above 1, twice differentiable (needed when the *output* is differentiated, as in physics-informed networks). It is f' , not f , that enters the backward recursion: it decides whether a gradient survives a layer.

Which activation? Measure before believing the folklore (Table 8.2)



Book Figure 8.7: a 64–30–30–30–10 network on the digits data, 60 epochs, mean $\pm$ s.d. over five seeds.

What the table says

Sigmoid 0.890 ± 0.010 , far behind, training loss two orders of magnitude higher: with three hidden layers the 1/4 per layer already bites. The seven non-saturating activations span 0.966–0.972 – *less than the spread between seeds* (0.004–0.008); tanh is nominally the best. With 360 test samples the standard error of an accuracy near 0.97 is 0.009 (Section 2.5): a difference of 0.006 was never resolvable. A paper declaring a winner from one run reports seed noise.

Working rule

ReLU first; leaky ReLU or ELU if units die; GELU/Swish in transformers, very deep networks, and wherever the output must be differentiated; tanh when a bounded activation is wanted; the logistic function in a hidden layer only to reproduce an old result. **The output activation is not a free choice:** identity for regression, sigmoid for two classes, softmax for K .

Pause and think 3: the derivative is what matters

Question

Ten sigmoid layers in a chain, every pre-activation near zero so that each σ' is at its maximum. By what factor is the gradient that reaches the first layer at most attenuated? And with ReLU? With a pre-activation of $z = 5$ in every sigmoid layer instead?

Pause and think 3: the derivative is what matters

Question

Ten sigmoid layers in a chain, every pre-activation near zero so that each σ' is at its maximum. By what factor is the gradient that reaches the first layer at most attenuated? And with ReLU? With a pre-activation of $z = 5$ in every sigmoid layer instead?

Answer

Each layer multiplies the error by $\sigma'(z') \leq 1/4$ (and by a weight), so at best $(1/4)^{10} \approx 10^{-6}$: the first layer receives a millionth of the signal and, at any sensible learning rate, does not move – the *vanishing gradient* of Section 8.9, and the reason the sigmoid sits alone at the bottom of Table 8.2. ReLU has derivative exactly one on the active side: the factor is one for every active unit and the signal arrives undiminished (the weights still scale it, which is the subject of initialisation, Section 8.10). At $z = 5$, $\sigma'(5) = 0.0066$ and ten layers give 10^{-22} : a saturated sigmoid network cannot learn at all, whatever the depth of the error at the output. We will see the mechanism with a single hidden neuron in twenty minutes.

The cost function and the optimisation problem (Section 8.7)

Collect every weight and bias into $\Theta = \{W^l, b^l\}_{l=1}^L$, Eqs. (8.31)–(8.32):

$$\hat{\Theta} = \arg \min_{\Theta} \frac{1}{n} \sum_{i=1}^n L(y_i, \tilde{y}_i(\Theta)) + \lambda \Omega(\Theta), \quad C(\Theta) = \frac{1}{2n} \sum_{i=1}^n (a_i^L - y_i)^2 \text{ (regression),}$$

the cross entropy (5.13)/(5.28) for classification – the loss is still decided by the distribution of the target, Section 3.6.

Three things we have never met before

Not convex: linear maps composed with non-linearities – local minima, saddles, and the answer depends on the initialisation; Section 4.1 no longer protects us. **Highly symmetric:** permute the hidden units and nothing changes, so every minimum has a large family of copies. **Very many parameters:** the Hessian of Section 4.2 is out of the question – first-order methods only, Sections 4.7–4.11.

What we are left with, and why it is enough

Gradient descent and the chain rule. That such networks train at all is an empirical fact, not a theorem (strict local minima are rare in high dimensions relative to saddles; the symmetric copies make *some* good solution easy to reach). The one thing we can make exact is the *gradient*: for the price of about two forward passes, whatever the number of parameters – reverse-mode AD, Section 4.14, written out by hand for a layered network. That is Part 3.

The smallest network: no hidden layer (Section 8.8.1, Eqs. 8.33–8.37)

One node per layer, so only the layer superscript survives: w^1, b^1, z^1, a^1 are numbers. Sigmoid activation, half-squared error for one pair (x, y) :

$$x \longrightarrow z^1 = w^1 x + b^1 \longrightarrow a^1 = \sigma(z^1) \longrightarrow C = \frac{1}{2}(a^1 - y)^2.$$

The neuron of Figure 5.4 with $p = 1$ and the loss swapped. C depends on w^1 only along $w^1 \rightarrow z^1 \rightarrow a^1 \rightarrow C$, so the chain rule (1.51):

$$\frac{\partial C}{\partial w^1} = \frac{\partial C}{\partial a^1} \frac{\partial a^1}{\partial z^1} \frac{\partial z^1}{\partial w^1} = (a^1 - y) \sigma'(z^1) x, \quad \frac{\partial C}{\partial b^1} = (a^1 - y) \sigma'(z^1)$$

(the same path, with $\partial z^1 / \partial b^1 = 1$ as its last factor).

Name the shared part: the error,
Eq. (8.35)

$$\delta^1 = (a^1 - y) \sigma'(z^1) = \frac{\partial C}{\partial z^1}, \quad \frac{\partial C}{\partial w^1} = \delta^1 x, \quad \frac{\partial C}{\partial b^1} = \delta^1,$$

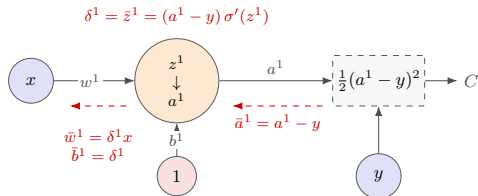
and one step of gradient descent, Eq. (8.37):
 $w^1 \leftarrow w^1 - \gamma \delta^1 x$, $b^1 \leftarrow b^1 - \gamma \delta^1$ – error times the signal that entered the parameter.

The delta rule once more – with one difference

For the cross entropy $\partial C / \partial z$ was the bare residual $\hat{p} - y$ (5.36): the softplus derivative cancelled σ' . With the squared error nothing cancels and the residual is multiplied by $\sigma'(z^1) \leq 1/4$, which dies once $|z^1|$ is a few units. A *saturated neuron learns slowly however wrong it is*: the factor f' that will sit in front of every error, and the seed of Section 8.9.

The same neuron read backwards: δ is the adjoint of z (Eqs. 8.38–8.39)

no hidden layer, Eq. (8.33)



Book Figure 8.10, top panel: black the forward pass, red dashed the reverse sweep.

Parameters as inputs, $v_{-1} = b^1$, $v_0 = w^1$; data as constants. The trace, Eq. (8.38):

$$\begin{aligned} v_1 &= v_0 x, & v_2 &= v_1 + v_{-1} = z^1, \\ v_3 &= \sigma(v_2) = a^1, & v_4 &= \frac{1}{2}(v_3 - y)^2 = C. \end{aligned}$$

Reverse sweep, Eq. (4.87), from $\bar{v}_4 = 1$, Eq. (8.39):

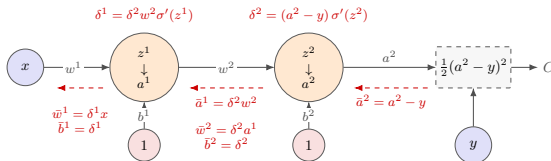
$$\begin{aligned} \bar{v}_3 &= v_3 - y = a^1 - y, & \bar{v}_2 &= \bar{v}_3 \sigma'(v_2) = \delta^1, \\ \bar{v}_0 &= \bar{v}_2 x = \delta^1 x, & \bar{v}_{-1} &= \bar{v}_2 = \delta^1. \end{aligned}$$

Read it

The error δ^1 is $\bar{z}^1$, exactly as $\bar{z} = \hat{p} - y$ was in Figure 5.5, and Eq. (8.36) is what the sweep delivers at the two inputs. One sweep, both gradients; forward mode (4.81) would need one per parameter. Backpropagation is this sweep continued through as many layers as the network has.

One hidden layer: the recursion appears (Eqs. 8.40–8.43)

one hidden layer, Eq. (8.40)



Book Figure 8.10, bottom panel. Eq. (8.40):

$$\begin{aligned} z^1 &= w^1 x + b^1, & a^1 &= \sigma(z^1), \\ z^2 &= w^2 a^1 + b^2, & a^2 &= \sigma(z^2), \\ C &= \frac{1}{2}(a^2 - y)^2. \end{aligned}$$

The forward pass computes *and stores* (z^1, a^1, z^2, a^2) : every one is needed on the way back.

Output layer = the previous example with a^1 in place of x , Eq. (8.41):

$$\delta^2 = (a^2 - y) \sigma'(z^2), \quad \frac{\partial C}{\partial w^2} = \delta^2 a^1, \quad \frac{\partial C}{\partial b^2} = \delta^2.$$

Hidden weight: the longer path $w^1 \rightarrow z^1 \rightarrow a^1 \rightarrow z^2 \rightarrow a^2 \rightarrow C$, Eq. (8.42):

$$\frac{\partial C}{\partial w^1} = \underbrace{(a^2 - y) \sigma'(z^2)}_{\delta^2} w^2 \sigma'(z^1) x.$$

The first two factors are δ^2 , *already computed*. With $\delta^1 = \partial C / \partial z^1$ as before, Eq. (8.43):

$$\boxed{\delta^1 = \delta^2 w^2 \sigma'(z^1)} \quad \frac{\partial C}{\partial w^1} = \delta^1 x, \quad \frac{\partial C}{\partial b^1} = \delta^1.$$

The pattern – and the reverse sweep that did not stop

Forwards and backwards

Forwards, every layer computes $z^l = w^l a^{l-1} + b^l$ and $a^l = \sigma(z^l)$. Backwards, every layer computes its error from the error downstream, times the weight that connects them, times its own $\sigma'(z^l)$; the gradient of a weight is that error times the activation that entered the weight, the gradient of a bias is the error itself. *Nothing is recomputed*: the long product of Eq. (8.42) is never formed, its first half is δ^2 and was already evaluated for the output layer. With ten hidden neurons in a chain each error costs one multiplication, each gradient one more.

In the language of Section 4.14

Eq. (8.43) is the reverse sweep of Eq. (8.39) that did not stop at the pre-activation. In Section 5.8.2 the sweep reached $\bar{z} = \hat{p} - y$ and stepped onto the edges carrying the parameters, because the inputs of the neuron were constants. Here the input a^1 of the output neuron is the output of a node, so the sweep continues: $\bar{a}^1 = \delta^2 w^2$ by the local derivative $\partial z^2 / \partial a^1 = w^2$, then $\bar{z}^1 = \bar{a}^1 \sigma'(z^1) = \delta^1$, and only then the edges w^1, b^1 . Pause and think 1, answered – and the whole of backpropagation, for one node per layer.

Numbers, and the program that is Eqs. (8.36)–(8.43) transcribed

```
def grad_one(w1, b1, x, y): # no hidden layer
    z1 = w1*x + b1; a1 = sigma(z1) # forward, store z1, a1
    delta1 = (a1 - y)*dsigma(z1) # Eq. (8.35)
    return 0.5*(a1-y)**2, delta1*x, delta1, delta1

def grad_two(w1, b1, w2, b2, x, y): # one hidden layer
    z1 = w1*x + b1; a1 = sigma(z1) # forward pass
    z2 = w2*a1 + b2; a2 = sigma(z2)
    delta2 = (a2 - y)*dsigma(z2) # output error, Eq.
    (8.41)
    delta1 = delta2*w2*dsigma(z1) # Eq. (8.43)
    return (0.5*(a2-y)**2, delta1*x, delta1,
           delta2*a1, delta2, delta1, delta2)
```

```
no hidden layer : C = 0.0362 delta1 = -0.0529
                  dC/dw1 = -0.1058 dC/db1 = -0.0529
one hidden layer: C = 0.0979 delta2 = -0.1092 delta1 =
                  -0.0215
                  dC/dw1 = -0.0429 dC/db1 = -0.0215
                  dC/dw2 = -0.0798 dC/db2 = -0.1092
```

At the point of Section 5.8.2, $w^1 = 0.25$, $b^1 = 0.5$, $x = 2$, $y = 1$: $z^1 = 1$, $a^1 = 0.731$, $C = 0.0362$, $\sigma'(z^1) = 0.197$, $\delta^1 = -0.0529$, gradient $(-0.106, -0.0529)$.

Compare with the cross entropy

At the same point the cross entropy gave $(-0.538, -0.269)$ (Part 1). The ratio is exactly $\sigma'(z^1) = 0.197$: the factor that the squared error does not cancel.

Add the output neuron, $w^2 = 1$, $b^2 = -0.5$

$z^2 = 0.231$, $a^2 = 0.557$, $C = 0.0979$, $\delta^2 = -0.109$, and by the boxed recursion $\delta^1 = \delta^2 w^2 \sigma'(z^1) = -0.0215$. Four gradients, one forward pass, one backward pass, and every number reused.

A first training program: the same equations with arrays (Section 8.8.1)

```
def forwardpropagation(x):
    z_1 = x @ w_1 + b_1 # hidden pre-activation
    a_1 = sigmoid(z_1)
    z_2 = a_1 @ w_2 + b_2 # output pre-activation
    return a_1, z_2 # linear output: a_2 = z_2

def backpropagation(x, y):
    a_1, a_2 = forwardpropagation(x)
    delta_2 = a_2 - y # linear output: f' = 1
    delta_1 = (delta_2 @ w_2.T) * a_1 * (1 - a_1) # Eq.
    (8.43)
    dW2 = a_1.T @ delta_2; db2 = np.sum(delta_2, axis=0)
    dW1 = x.T @ delta_1; db1 = np.sum(delta_1, axis=0)
    return dW2, db2, dW1, db1, 0.5*np.sum((a_2 - y)**2)

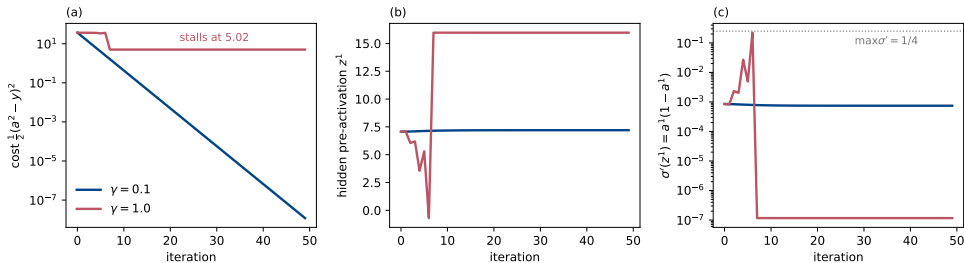
np.random.seed(0) # w_1, w_2 ~ randn(1, 1); biases 0.01
x = np.array([[4.0]]); y = 2*x + 1.0 # one sample, target 9
gamma = 0.1
for i in range(50):
    dW2, db2, dW1, db1, cost = backpropagation(x, y)
    w_2 -= gamma*dW2; b_2 -= gamma*db2
    w_1 -= gamma*dW1; b_1 -= gamma*db1
```

```
iteration 0: cost = 36.895631
iteration 10: cost = 0.426990
iteration 20: cost = 0.004939
iteration 49: cost = 0.000000
w_1 = [1.7958] b_1 = [0.0179]; w_2 =
      [4.6950] b_2 = [4.3084]
```

Two details that will survive unchanged

Samples are rows (shape (1, 1) here), weights have shape (inputs, outputs), the forward pass reads $x @ w_1 + b_1$, Eq. (8.21). The output is *linear*, so $\delta^2 = a^2 - y$ with no $f' =$ Eq. (8.69), the natural pairing of a regression target with the squared error. $\text{delta_2} @ w_2.T$ pulls the error back through the weight, $x.T @ \text{delta_1}$ is the outer product of inputs and errors: for scalars the multiplications of Eq. (8.43), but Eqs. (8.66) and (8.68) of next week without a change to the code. Not learning in any interesting sense – four parameters, one point – but every line survives into the 2–2–1 network and into the class of Section 8.13.

The learning rate matters even here: a saturated hidden neuron (week40.ipynb)



The program of the previous frame at $\gamma = 0.1$ (blue) and $\gamma = 1$ (red). **(a)** At $\gamma = 0.1$ the cost falls by nine orders of magnitude in fifty iterations. At $\gamma = 1$ the first steps drive the hidden weight to $w^1 \approx 4$, so $z^1 \approx 16$ **(b)**, and $\sigma'(z^1) \approx 10^{-7}$ **(c)**: every gradient reaching the first layer, $\delta^1 x = \delta^2 w^2 \sigma'(z^1) x$, is multiplied by that factor, the hidden neuron stops learning, and the cost stalls at 5.02 for the remaining forty iterations however large the error at the output. This is the mechanism of Section 8.9 in a program of twenty lines. Note the blue curve in (c) too: $\sigma'(z^1) \approx 7 \times 10^{-4}$ throughout – see the next question.

Pause and think 4: who did the learning?

Question

In the successful run ($\gamma = 0.1$) the hidden weight started at $w^1 = 1.764$ (the first randn of seed 0) and ended at 1.796, while $\sigma'(z^1)$ stayed near 7×10^{-4} . Which parameters fitted the target 9, and why does the network still reach a cost of 10^{-8} ? What would happen with a second training point at $x = -4$, $y = -7$?

Pause and think 4: who did the learning?

Question

In the successful run ($\gamma = 0.1$) the hidden weight started at $w^1 = 1.764$ (the first randn of seed 0) and ended at 1.796, while $\sigma'(z^1)$ stayed near 7×10^{-4} . Which parameters fitted the target 9, and why does the network still reach a cost of 10^{-8} ? What would happen with a second training point at $x = -4$, $y = -7$?

Answer

The output layer did all of it: with $a^1 = \sigma(z^1) \approx 0.9993$ essentially constant, $a^2 = w^2 a^1 + b^2$ is a linear function of two free parameters (w^2, b^2), and any single target can be hit with the hidden neuron frozen – w^2 went from 0.400 to 4.695 and b^2 from 0.01 to 4.308. The gradient reaching w^1 was $\delta^2 w^2 \sigma'(z^1) x \sim 10^{-3} \times$ the output error, so it hardly moved. With a second point at $x = -4$ the hidden neuron would have to output two *different* values, $a^1(4) \neq a^1(-4)$, and the frozen unit at 0.9993 for $x = 4$ would give $\sigma(-7.0) \approx 10^{-3}$ for $x = -4$: fine by luck for this sign pattern, but in general the hidden layer must *learn*, and a saturated one cannot. Two lessons for the class of Section 8.13: initialise small (Section 8.10) so that z^1 starts in the region where σ' is not tiny, and watch f' , not only the cost.

The same by automatic differentiation: we write the forward pass only

```
def C_one(p, x, y): # forward pass, single neuron
    return 0.5*(jax.nn.sigmoid(p[0]*x + p[1]) - y)**2
def C_two(p, x, y): # ... with the hidden neuron
    a1 = jax.nn.sigmoid(p[0]*x + p[1])
    return 0.5*(jax.nn.sigmoid(p[2]*a1 + p[3]) - y)**2

p1, p2 = jnp.array([0.25, 0.5]), jnp.array([0.25, 0.5, 1.0, -0.5])
print(jax.grad(C_one)(p1, 2.0, 1.0)) # [-0.1058 -0.0529]
print(jax.grad(C_two)(p2, 2.0, 1.0)) # [-0.0429 -0.0215 -0.0798
    -0.1092]
```

```
def cost(params, x, y):
    w_1, b_1, w_2, b_2 = params
    a_1 = jax.nn.sigmoid(x @ w_1 + b_1) # the forward pass,
    nothing else
    return 0.5*jnp.sum((a_1 @ w_2 + b_2 - y)**2)
grad = jax.jit(jax.grad(cost)) # the backward pass, generated
for i in range(50): # same costs, same parameters
    g = grad(params, x, y)
    params = [p - gamma*gp for p, gp in zip(params, g)]
```

Every digit agrees with `grad_one/grad_two`; `jax.jvp` with the two unit seeds (notebook) returns the same two numbers one at a time.

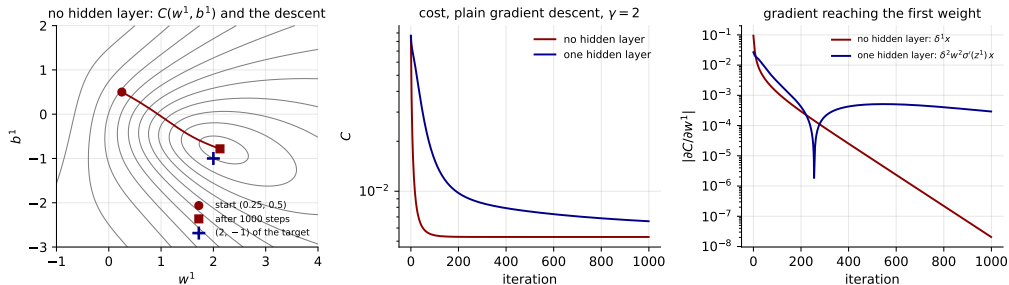
What happened

Nothing in the listing mentions δ , σ' or the chain rule. `jax.grad` traced the forward pass, recorded the tape (z^1, a^1, z^2, a^2) and swept it in reverse – Eq. (8.39) continued through the hidden neuron. The training program loses backpropagation and prints the *same* costs and parameters.

Why we still write it by hand

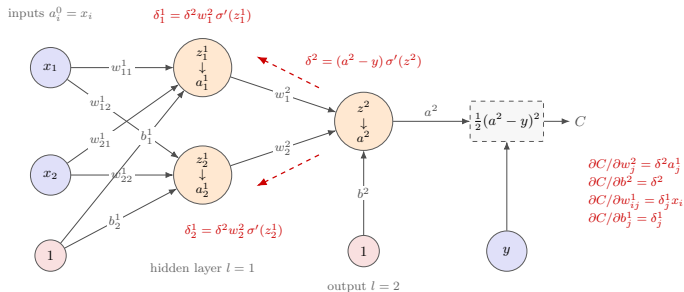
The pattern of every framework from Chapter 10 on: we write the forward pass, the library the backward one. The hand-written version (Section 8.13) is the only way to know what the library does; the gradient check of Eq. (4.102) verifies that the two agree. Project 2 asks for both.

Training the two networks on twenty points (book Figure 8.11)



$n = 20$ inputs in $[-2, 2]$, $y_i = \sigma(2x_i - 1) + \varepsilon_i$, $\varepsilon_i \sim N(0, 0.1^2)$, cost averaged over the sample (the errors δ^l of the observations simply add), 1000 steps of plain gradient descent with $\gamma = 2$ from the parameters above. **Left:** the cost surface of the single neuron is *not convex* – the sigmoid sees to that – but has one basin around $(2, -1)$; the path from $(0.25, 0.5)$ ends at $(2.13, -0.78)$, $C = 0.0053$. **Middle:** the network with a hidden neuron contains the single one as a special case and has four parameters, yet reaches only $C = 0.0066$ in the same number of steps. **Right:** why – the gradient reaching w^1 is $\delta^1 x$ in both, but in the deeper network δ^1 carries the extra factor $w^2 \sigma'(z^2) \approx 0.25$ of Eq. (8.43), which shrinks the first-layer gradient by that amount. Every gradient was again compared with `jax.grad` at every iteration: largest discrepancy 3×10^{-17} .

Two inputs, two hidden nodes, one output (Section 8.8.2, book Figure 8.12)



The smallest network in which a weight needs two indices, Eqs. (8.44)–(8.47): w_{ij}^1 leaves input i and arrives at hidden node j ,

$$z_j^1 = w_{1j}^1 x_1 + w_{2j}^1 x_2 + b_j^1, \quad a_j^1 = \sigma(z_j^1), \quad j = 1, 2; \quad z^2 = w_1^2 a_1^1 + w_2^2 a_2^1 + b^2, \quad a^2 = \sigma(z^2), \quad C = \frac{1}{2}(a^2 - y)^2,$$

or $\mathbf{z}^1 = (\mathbf{W}^1)^T \mathbf{x} + \mathbf{b}^1$ with $\mathbf{W}^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 \\ w_{21}^1 & w_{22}^1 \end{bmatrix}$, Eq. (8.45). Nine parameters; the forward pass stores $z_1^1, z_2^1, a_1^1, a_2^1, z^2, a^2$. This is the last time we write the derivatives one by one.

The nine derivatives, three at a time (Eqs. 8.48–8.52)

Output layer – the scalar case with the input activation carrying the index of its edge, Eq. (8.48):

$$\delta^2 = \frac{\partial C}{\partial z^2} = (a^2 - y) \sigma'(z^2), \quad \frac{\partial C}{\partial w_j^2} = \delta^2 a_j^1 \quad (j = 1, 2), \quad \frac{\partial C}{\partial b^2} = \delta^2.$$

Hidden layer – take w_{11}^1 : it affects z_1^1 only, which reaches C only through z^2 along the edge w_1^2 ; the same five-factor chain as Eq. (8.42), Eq. (8.49):

$$\frac{\partial C}{\partial w_{11}^1} = \underbrace{\frac{\partial C}{\partial a^2} \frac{\partial a^2}{\partial z^2}}_{\delta^2} \frac{\partial z^2}{\partial a_1^1} \frac{\partial a_1^1}{\partial z_1^1} \frac{\partial z_1^1}{\partial w_{11}^1} = \delta^2 w_1^2 \sigma'(z_1^1) x_1.$$

w_{21}^1 (from x_2 , same node) has the identical chain with x_2 last; b_1^1 has the factor 1. So each hidden node has an error of its own, Eqs. (8.50)–(8.51), and collected, Eq. (8.52):

$$\delta_j^1 = \delta^2 w_j^2 \sigma'(z_j^1), \quad \frac{\partial C}{\partial w_{ij}^1} = \delta_j^1 a_i^0, \quad \frac{\partial C}{\partial b_j^1} = \delta_j^1, \quad i, j \in \{1, 2\}.$$

The rule, now with indices (Eq. 8.53)

$\partial C / \partial w_{ij}^l = \delta_j^l a_i^{l-1}$, $\partial C / \partial b_j^l = \delta_j^l$: the gradient of a weight is the error at the node it arrives at, times the signal at the node it leaves. The error carries the receiving index, the activation the sending index, and the gradient has the shape of the weight matrix.

Vector form, outer product – and the fan-out the sweep did not need (8.54–8.56)

The two hidden errors as a vector, Eq. (8.54),

$$\delta^1 = \begin{bmatrix} \delta_1^1 \\ \delta_2^1 \end{bmatrix} = (W^2 \delta^2) \circ \sigma'(z^1), \quad W^2 = \begin{bmatrix} w_1^2 \\ w_2^2 \end{bmatrix},$$

and the four hidden weight gradients as an outer product, Eq. (8.55),

$$\frac{\partial C}{\partial W^1} = x(\delta^1)^T = \begin{bmatrix} x_1 \delta_1^1 & x_1 \delta_2^1 \\ x_2 \delta_1^1 & x_2 \delta_2^1 \end{bmatrix},$$

entry (i, j) in the position of w_{ij}^1 . These are, for this network, Eqs. (8.66) and (8.68) of next week. Update, Eq. (8.56): $w_{ij}^l \leftarrow w_{ij}^l - \gamma \delta_j^l a_i^{l-1}$, $b_j^l \leftarrow b_j^l - \gamma \delta_j^l$, all nine at once.

Read as the reverse sweep of Section 4.14.4

New: the adjoint δ^2 leaves the output node along *two* edges, w_1^2 and w_2^2 – a fan-in seen from the output; it costs nothing, each hidden node receives its own share. **Still missing:** the fan-out. Each hidden node feeds *one* node downstream, so its adjoint arrives in a single term and no sum is needed. (The inputs x_i do feed both hidden nodes, but they are constants of the trace and we never need their adjoints.)

The only thing the general case adds

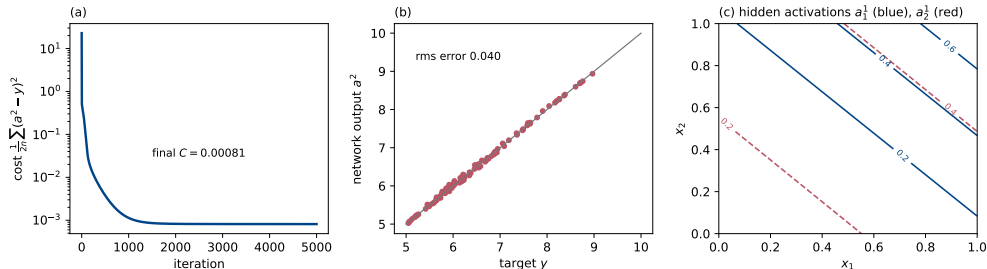
Give the network a second output node: z_j^1 reaches C along two paths, the adjoints from the two outputs must be *added* at each hidden node, and Eq. (8.52) acquires $\sum_k \delta_k^2 w_{jk}^2$ – the fan-out rule, exactly as the two branches of Figure 5.5 were summed at v_2 . That sum, Eq. (8.64), is Section 8.8.3 – next week.

Code: the 2-2-1 network, five lines of backward pass, checked with AD

```
def forward(X, W1, b1, W2, b2):
    A1 = sigmoid(X @ W1 + b1) # hidden layer, samples in rows
    return A1, A1 @ W2 + b2 # linear output node
def backward(X, y, W1, b1, W2, b2):
    n = X.shape[0]
    A1, A2 = forward(X, W1, b1, W2, b2)
    delta2 = (A2 - y)/n # output error, f' = 1, Eq. (8.48)
    delta1 = (delta2 @ W2.T) * A1*(1 - A1) # Eq. (8.54): (W2 delta2) o sigma'(z1)
    dW2 = A1.T @ delta2; db2 = delta2.sum(axis=0) # Eq. (8.53), l = 2
    dW1 = X.T @ delta1; db1 = delta1.sum(axis=0) # Eq. (8.55), summed over the samples
    return dW1, db1, dW2, db2, 0.5*np.sum((A2 - y)**2)/n
def cost(params, X, y): # the same model, for jax.grad
    W1, b1, W2, b2 = params
    return 0.5*jnp.mean((jax.nn.sigmoid(X @ W1 + b1) @ W2 + b2 - y)**2)
g_hand = backward(X, y, W1, b1, W2, b2)[:4] # W1 (2, 2), W2 (2, 1) ~ N(0, 1), seed 41
g_jax = jax.grad(cost)([jnp.array(p) for p in (W1, b1, W2, b2)], X, y)
print(max(float(jnp.max(jnp.abs(h - j))) for h, j in zip(g_hand, g_jax))) # 1.78e-15
```

X: $n = 100$ rows, two columns; W^1 is 2×2 , W^2 is 2×1 . Every line of backward is one of Eqs. (8.48), (8.54), (8.55) for all samples at once, $X.T @ \text{delta1}$ summing the outer products $x_i(\delta_i^1)^T$. Linear output, target $y = x_1^2 + 3x_1x_2 + x_2^2 + 5$, $\gamma = 0.5$, 5000 iterations: cost $22.76 \rightarrow 0.00113 \rightarrow 0.00081$ after 0/1000/5000, $W^1 = \begin{bmatrix} 2.524 & 1.048 \\ 2.558 & 1.052 \end{bmatrix}$, $b^1 = (-4.12, -1.97)$, $W^2 = (5.08, 2.98)$, $b^2 = 4.45$. The class of Section 8.13 has these five lines inside a loop over layers.

The 2-2-1 network at work (week40.ipynb)



(a) The cost falls from 22.8 to 10^{-3} in a thousand iterations and then creeps to $C = 0.00081$ – a mean squared error of 1.6×10^{-3} , rms error 0.04 on targets between 5 and 10 **(b)**. **(c)** The two learned features: both hidden nodes are sigmoids of a linear function of (x_1, x_2) – ridges across the square, a_1^1 steep along the diagonal, a_2^1 gentler – and the linear output node combines them, $a^2 = 5.08 a_1^1 + 2.98 a_2^1 + 4.45$, into a surface that follows $x_1^2 + 3x_1x_2 + x_2^2 + 5$ to 0.4% . Two hidden nodes, nine parameters, no polynomial features chosen by us. What limits the fit is the two-ridge shape; more hidden nodes would add more ridges – the constructive theorem of Section 8.4.1 in two dimensions.

Pause and think 5: a second output node

Question

Give the 2-2-1 network a second output node, $z_k^2 = \sum_j w_{jk}^2 a_j^1 + b_k^2$ for $k = 1, 2$, with the cost $\frac{1}{2} \sum_k (a_k^2 - y_k)^2$. Which of the nine-plus-three derivatives change their form, and what is the new expression for δ_1^1 ? How does this read in the language of the reverse sweep, and what is the vector form?

Pause and think 5: a second output node

Question

Give the 2–2–1 network a second output node, $z_k^2 = \sum_j w_{jk}^2 a_j^1 + b_k^2$ for $k = 1, 2$, with the cost $\frac{1}{2} \sum_k (a_k^2 - y_k)^2$. Which of the nine-plus-three derivatives change their form, and what is the new expression for δ_1^1 ? How does this read in the language of the reverse sweep, and what is the vector form?

Answer

The output layer is unchanged in form: $\delta_k^2 = (a_k^2 - y_k)\sigma'(z_k^2)$, $\partial C / \partial w_{jk}^2 = \delta_k^2 a_j^1$, one error per output node. What changes is the hidden error. z_1^1 now reaches C along *two* paths, through z_1^2 via w_{11}^2 and through z_2^2 via w_{12}^2 , and the chain rule adds them: $\delta_1^1 = (\delta_1^2 w_{11}^2 + \delta_2^2 w_{12}^2)\sigma'(z_1^1) = (\sum_k \delta_k^2 w_{1k}^2)\sigma'(z_1^1)$. In the sweep: a_1^1 has two successors, so its adjoint is the *sum* of the two contributions – the fan-out rule of Section 4.14.4 that the 2–2–1 network never needed. Vector form: $\delta^1 = (W^2 \delta^2) \circ \sigma'(z^1)$ with W^2 now 2×2 – Eq. (8.54) with a matrix in place of a column, which is Eq. (8.66), the backpropagation equation, in full generality. The weight gradients keep the outer-product form, Eq. (8.68). That is all next week adds.

Next week: the general case, the four equations, your own code (8.8.3–8.13)

The four equations, Eqs. (8.65)–(8.68)

$$\delta^L = f'(z^L) \circ \partial C / \partial a^L \quad (\text{start at the output})$$

$$\delta^l = (W^{l+1} \delta^{l+1}) \circ f'(z^l) \quad (\text{carry it back})$$

$$\partial C / \partial b_j^l = \delta_j^l, \quad \partial C / \partial w_{ij}^l = \delta_j^l a_i^{l-1} \quad (\text{the gradients})$$

The second is Eq. (8.64): the same graph traversed backwards with the transposed weights (book Figure 8.13); the sum over k is the fan-out we just met. For the natural output/loss pairings the first collapses to $\delta^L = a^L - y$, Eq. (8.69).

Cost

One matrix–vector product per layer, like the forward pass: the whole gradient for about two forward passes whatever the number of parameters – Prop. 4.2 for a layered network. Finite differences, Eq. (4.102): one pass per parameter; for 10^6 weights a second against a fortnight.

The algorithm (Section 8.8.5) and the class (Section 8.13)

Forward pass storing z^l, a^l ; output error; backward pass $l = L - 1, \dots, 1$; gradients; update with any optimiser of Chapter 4, on minibatches. Then a `NeuralNetwork` class: a loop over layers, any activation, both output heads, and its gradient check at 10^{-6} – 10^{-8} .

And what makes it work

Vanishing/exploding gradients (8.9), initialisation (8.10), classification heads (8.11), regularisation and early stopping (8.12). Project 2 asks for exactly this network, on the regression problem of Project 1 and on a classification problem.

Summary of week 40

- ▶ **The reminder:** logistic regression is one neuron; its learning rule is error times the input on the edge (5.36–5.37); read as a computational graph, one reverse sweep gives the whole gradient for 1.4–2 evaluations of the cost whatever p , forward mode needs p sweeps (5.40–5.42). Training asks the reverse-mode question.
- ▶ **Networks:** a neuron whose inputs are outputs of other neurons; one layer draws one line and cannot do XOR, a hidden layer learns the features that make the problem linear; non-linear f is essential, and *not a polynomial* is the exact condition for universality (8.4) – an existence statement that says nothing about width, weights or learnability. Only f' enters the backward pass: sigmoid $\leq 1/4$ per layer, ReLU 1. The cost (8.31) is non-convex, symmetric and huge: first-order methods, and an exact gradient.
- ▶ **Backpropagation from the smallest networks:** the error $\delta = \partial C / \partial z$ is the adjoint of the pre-activation (8.35, 8.39); a weight's gradient is δ times its input; one hidden layer gives the recursion $\delta^1 = \delta^2 w^2 \sigma'(z^1)$ (8.43) – the reverse sweep that does not stop; the 2–2–1 network fixes the indices, $\partial C / \partial w_{ij}^l = \delta_j^l a_i^{l-1}$ (8.53), vector and outer-product form (8.54–8.55); `jax.grad` of the forward pass reproduces every digit; a saturated hidden neuron stops learning (the $\gamma = 1$ run).
- ▶ **What is left:** the sum over receiving nodes when a node feeds several – the fan-out rule – and with it the four equations (8.65–8.68), the algorithm and the class. Next Monday.

Tomorrow and the deadline

Tuesday is *only* Project 1: your questions, then two cases on reading, analysing and presenting your own results. No weekly exercises. **Project 1 is due Monday October 5.**